

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Fundamentación de Robótica	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez	
Módulo:	Actividad: Actividad 2.2 PWM con ESP32	
Fecha: 03 de marzo del 2026		

Resumen

En esta actividad relacionada con el mini reto 3 se desarrolló e implementó un sistema de control de velocidad y dirección para un motor de corriente directa utilizando un microcontrolador la cual fue un ESP32, un potenciómetro como una entrada analógica y un puente H para el control del sentido de giro del motor. De igual manera la lectura de la entrada analógica del potenciómetro es obtenida mediante el convertidor analógico digital de la ESP32 y es posteriormente convertida en una señal PWM de 8 bits con una frecuencia de 5000 Hz, permitiendo regular el duty cycle y con ello la velocidad del motor.

El sistema fue realizado en micro-ROS con el fin de permitir la comunicación entre nodos mediante la publicación y suscripción de tópicos. Es por ello que como parte de la arquitectura implementada se desarrollaron publishers encargados de transmitir variables del sistema como el duty cycle, el voltaje recibido, el valor PWM generado y el estado del motor, mientras que con un subscriber permitimos que el sistema reciba comandos externos para controlar el sentido del giro del motor. Asimismo, en la realización de esta práctica se identificó y corrigió un error en la conversión del ADC relacionado con la resolución de 12 bits de la ESP32, lo cual provocaba inconsistencias entre la lectura analógica y la señal observada en un osciloscopio. Los resultados obtenidos de esta actividad mostraron una correspondencia adecuada entre la señal y la respuesta del motor, confirmando el correcto funcionamiento del sistema implementado.

Objetivos

El objetivo principal de esta actividad fue diseñar e implementar un sistema de control de velocidad y dirección para un motor de corriente directa usando una ESP32 y un puente H con el fin de integrarlo e implementarlo con micro-ROS y validar su funcionamiento mediante un osciloscopio y usando `rqt_plot` para observar la señal de pwm generada. De igual manera, otro de los objetivos de esta práctica fue implementar un potenciómetro empleando el convertidor analogico-digital de la ESP32 con el propósito de transformar la variación de voltaje en un valor digital. A partir de dicha lectura, se estableció la conversión de la señal analógica en una señal PWM de 8 bits, permitiendo regular el duty cycle y la velocidad del motor.

Otro de los objetivos de esta práctica fue controlar la dirección del giro del motor mediante el uso de un puente H, permitiendo que el motor girara en sentido derecho, izquierdo o se detuviera, todo ello a través de comandos recibidos por medio de un suscriptor en nuestro código de micro-ROS. De igual manera, se buscó identificar y documentar las variables del sistema como el duty cycle, el valor PWM en un rango de 0 a 255, el voltaje obtenido y el estado de giro en el que se encontraba el motor. Finalmente, uno de los objetivos principales de esta práctica fue analizar la señal de PWM con un osciloscopio y comparar la variación física del potenciómetro con la respuesta gráfica obtenida mediante `rqt_plot`.

Introducción

Para esta actividad se implementó un programa encargado de controlar la velocidad de un motor DC, con ayuda de un PWM (Pulse With Modulation), usando un Puente H. El PWM permite controlar una señal, simulando una señal analógica mediante pulsos digitales rápidos, la variación del tiempo en que la señal permanece activa se llama duty cycle, o en español, ciclo de trabajo, y esta variación permite regular energía, atenuar luces o en este caso, variar la velocidad de giro del motor. Este programa se desarrolló en micro-ROS para poder programar el MCU, que en este caso fue una ESP32, que necesita principalmente la configuración de la frecuencia y la señal del duty cycle. Igualmente, para esta práctica fue útil utilizar un osciloscopio, pues se utilizó para ver la señal de PWM y estar seguros de que efectivamente se está mandando y recibiendo la señal.

Finalmente, micro-ROS es una extensión de ROS, sin embargo, esta es dedicada específicamente para microprocesadores de recursos limitados, además permite la integración

de sensores y algunos actuadores, permite la comunicación de forma serial, Wi-Fi e incluso Ethernet [1].

Solución problema

Migración a micro-ROS:

La solución al problema planteado consistió en migrar un sistema de control de velocidad de motor DC basado en PWM nativo del ESP32 hacia una arquitectura distribuida usando micro-ROS. El sistema original operaba de manera monolítica en el ESP32, realizando:

- Lectura analógica de potenciómetro (ADC)
- Cálculo de duty cycle basado en voltaje
- Generación de señal PWM
- Control de dirección mediante puente H
- Visualización por monitor serial

Limitaciones identificadas: El sistema no permitía integración con otros nodos ROS2, carecía de capacidad de control remoto, y no publicaba sus variables para monitoreo externo.

Se identificaron las siguientes variables del sistema que debían ser publicadas en topics ROS2:

Variable	Tipo	Descripción
Duty Cycle	Float32	Porcentaje de ciclo útil (0-100%)
Voltaje	Float32	Voltaje del potenciómetro (0-3.3V)
PWM Raw	Float32	Valor PWM discreto (0-255)
Estado Motor	String	Dirección: "Derecha", "Izquierda", "Detenido"

Identificación del error: El código original contenía un error crítico en la conversión ADC. El ESP32 cuenta con un ADC de 12 bits de resolución, lo que genera valores en el rango de 0 a 4095, utilizar una resolución incorrecta (10 bits = 1023 o 8 bits = 255) cuando el ESP32 usa 12 bits provocaba que:

- El voltaje calculado fuera superior al real

- La señal PWM en el osciloscopio mostrara valores incorrectos (se “reiniciaba”)
- El duty cycle reportado no coincidiera con la posición del potenciómetro
- La respuesta del motor fuera inconsistente con la entrada analógica

Solución:

```
voltaje = pot * (Vcc / 4095.0) - 12 bits
duty = 100.0 * voltaje / Vcc
pwm = map(pot, 0, 4095, 0, 255);
```

El ADC del ESP32 tiene una resolución de 12 bits por defecto, generando valores digitales entre 0 y 4095. Al dividir por 4095.0 se obtiene un factor de escala correcto que convierte la lectura digital a voltaje real (0-3.3V). Posteriormente, se mapea este rango al PWM de 8 bits (0-255) para el control del motor.

Publishers:

Se implementaron cuatro publishers para transmitir las variables del sistema:

Topic	Tipo Mensaje	Frecuencia
/motor/duty_cycle	std_msgs/Float32	10 Hz
/motor/voltaje	std_msgs/Float32	10 Hz
/motor/pwm_raw	std_msgs/Float32	10 Hz
/motor/status	std_msgs/String	On-demand

Implementación en código:

```
RCCHECK(rclc_publisher_init_default(
&pub_duty, &node,
ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
"motor/duty_cycle"));
```

Suscriber

Se implementó un suscriptor al topic motor/cmd (std_msgs/String) para recibir comandos de control externos. Comandos aceptados:

- Activar rotación a la derecha
- Activar rotación a la izquierda
- Detener motor (Stop)

Callback implementado:

```
void cmd_callback(const void* msgin) {
const std_msgs__msg__String* msg =
(const std_msgs__msg__String*)msgin;
char opcion = msg->data.data[0];
```

```

switch (opcion) {
    case 'D':
        motorEnMovimiento = true;
        derecha();
        break;
    case 'I':
        motorEnMovimiento = true;
        izquierda();
        break;
    case 'S':
        motorEnMovimiento = false;
        detener();
        break;
}

```

Funciones de control implementadas:

```

void derecha() {
    digitalWrite(In1, HIGH);
    digitalWrite(In2, LOW);
    status.data.data = "Derecha";
    RCSOFTCHECK(rcl_publish(&pub_status, &status, NULL));
}

```

```

void izquierda() {
    digitalWrite(In1, LOW);
    digitalWrite(In2, HIGH);
    status.data.data = "Izquierda";
    RCSOFTCHECK(rcl_publish(&pub_status, &status, NULL));
}

```

```

void detener() {
    digitalWrite(In1, LOW);
    digitalWrite(In2, LOW);
    ledcWrite(PWM1_Ch, 0);
    status.data.data = "Detenido";
    RCSOFTCHECK(rcl_publish(&pub_status, &status, NULL));
}

```

Control de dirección mediante puente H

El control de dirección del motor se implementó mediante un puente H (driver L298N o similar), utilizando dos pines de control digital:

In1 (GPIO23)	In2 (GPIO22)	PWM (GPIO26)	Acción
HIGH	LOW	Variable	Giro a la derecha
LOW	HIGH	Variable	Giro a la izquierda
LOW	LOW	0	Motor detenido

Configuración PWM del ESP32

La señal PWM se configuró utilizando el periférico LEDC (LED Controller) del ESP32:

Parámetros de configuración:

- Frecuencia: 5000 Hz
- Resolución: 8 bits — Genera 256 niveles de velocidad (0-255)
- Canal PWM: 0 — Canal LEDC asignado
- Pin de salida: GPIO 26

```
#define freq 5000
#define resolution 8
#define PWM1_Ch 0
```

```
ledcSetup(PWM1_Ch, freq, resolution);
ledcAttachPin(PWM_PIN, PWM1_Ch);
```

Sistema de Publicación Periódica

Se implementó un timer que ejecuta cada 100 ms (10 Hz) para publicar las variables del sistema:

```
RCCHECK(rclc_timer_init_default(
&timer, &support,
RCL_MS_TO_NS(100), // 100 ms = 10 Hz
timer_callback));
```

Lógica del timer callback:

```
void timer_callback(rcl_timer_t* timer, int64_t last_call_time) {
if (motorEnMovimiento) {
lecturaPWM(); // Lee potenciómetro y calcula variables
msg_duty.data = duty;
msg_voltaje.data = voltaje;
```

```

msg_pwm.data = (float)pwm;

// Publicar en topics correspondientes
RCSoftCHECK(rcl_publish(&pub_duty, &msg_duty, NULL));
RCSoftCHECK(rcl_publish(&pub_voltaje, &msg_voltaje, NULL));
RCSoftCHECK(rcl_publish(&pub_pwm, &msg_pwm, NULL));
}

```

Memoria dinámica:

Los mensajes de tipo String en micro-ROS requieren gestión manual de memoria:

```

// Reservar memoria para mensajes String
msg_cmd.data.data = (char*)malloc(10 * sizeof(char));
msg_cmd.data.size = 0;
msg_cmd.data.capacity = 10;
status.data.data = (char*)malloc(20 * sizeof(char));
status.data.size = 0;
status.data.capacity = 20;

```

Los mensajes std_msgs/String usan arrays dinámicos que deben ser inicializados antes de su uso para evitar errores de segmentación (segmentation fault).

Executor y Manejo de Callbacks

Se configuró un executor ROS con 2 handles:

1. Timer para publicación periódica (10 Hz)
2. Subscription para recepción de comandos

```

RCHECK(rclc_executor_init(&executor, &support.context, 2, &allocator));
RCHECK(rclc_executor_add_timer(&executor, &timer));
RCHECK(rclc_executor_add_subscription(&executor, &sub_cmd, &msg_cmd,
&cmd_callback, ON_NEW_DATA));

```

Loop principal:

```

void loop() {
RCSoftCHECK(rclc_executor_spin_some(&executor, RCL_MS_TO_NS(10)));

```

}

El ejecutor procesa callbacks cada 10 ms, permitiendo respuesta rápida a comandos mientras mantiene publicación periódica.

Resultados

Los resultados fueron una ejecución exitosa del agente de micro-ROS al conectarse con la ESP32 mediante puerto serial, y generar los tópicos `/duty_cycle`, `voltaje`, `pwm_raw` y `status`. El indicador más relevante del éxito del experimento fue la generación adecuada de la señal variable de PWM. La señal en concreto tiene una resolución de 8 bits y una frecuencia de 5000 Hz. Esto se pudo comprobar mediante la visualización de la misma a través de un osciloscopio, cuya punta estuvo monitoreando el pin 26 de la ESP32, por el cual salía la misma señal hasta llegar al pin *Enable A* del puente H al que estaba conectado el motor.

Asimismo, se puede comprobar el estatus de la señal al comparar las gráficas (generadas mediante `rqt_plot`) correspondientes a los tópicos de `/motor/duty_cycle` y `/motor/pwm_raw`. Se puede observar que al girar la posición del potenciómetro, el valor del duty cycle cambiaba en un valor del 0 al 100 (representando el porcentaje de tiempo de la señal en la cual el motor permanece activo). Por otro lado, el valor de PWM cambia en un intervalo de entre 0 a 255, representando la resolución de 8 bits que tiene la señal.

A continuación, se muestran los resultados de forma visual:

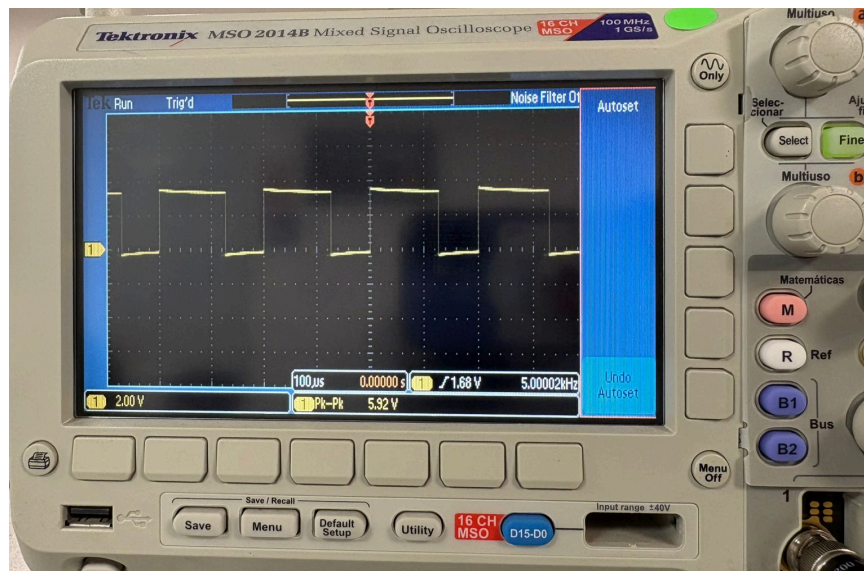


Imagen 1. Visualización del primer caso de la señal de PWM.

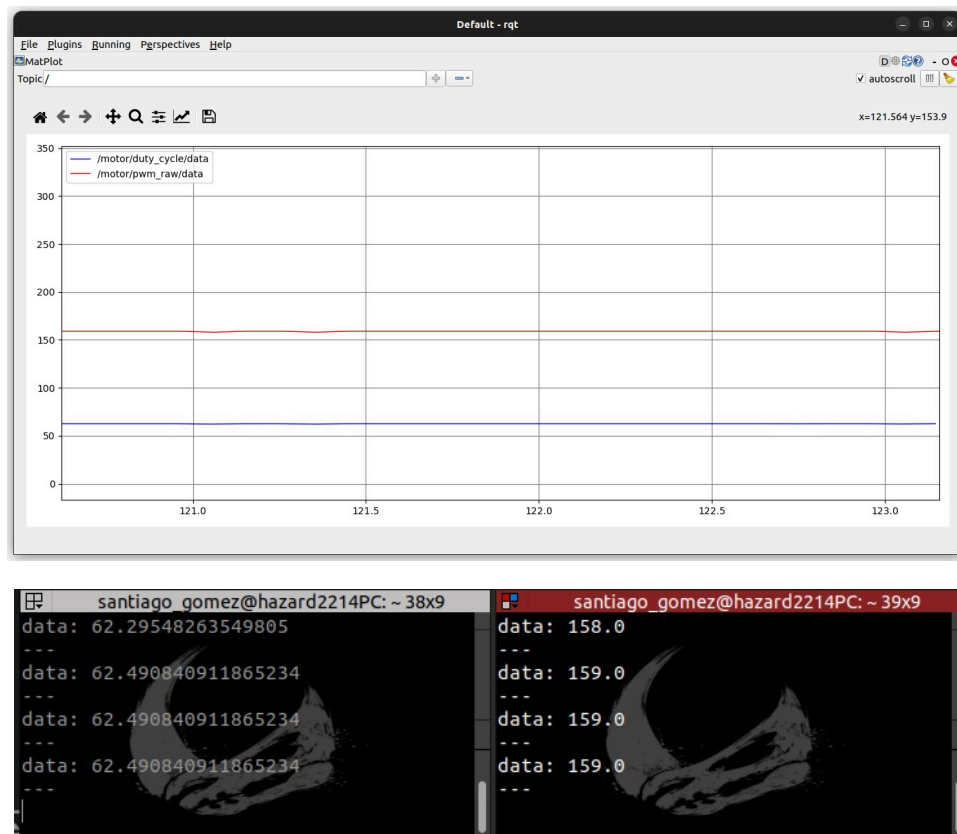


Imagen 2. Equivalencia en tópicos de ROS 2 de la señal PWM al primer caso.

En las imágenes **Imagen 1** e **Imagen 2**, se puede observar que la frecuencia en la que la señal opera es de $200\ \mu\text{s}$, lo cual es equivalente a $5000\ \text{Hz}$. Asimismo, el tiempo de estado alto de la señal es de alrededor de $125\ \mu\text{s}$, lo cual equivale a un duty cycle de aproximadamente un $62.5\ \%$, y a un valor de 159 en una escala de 0 a 255 . Estos mismos valores son los que se están mostrando tanto en la gráfica de `rqt_plot` como en la terminal al correr el comando `ros2 topic echo` a los tópicos `/motor/duty_cycle` y `/motor/pwm_raw`.

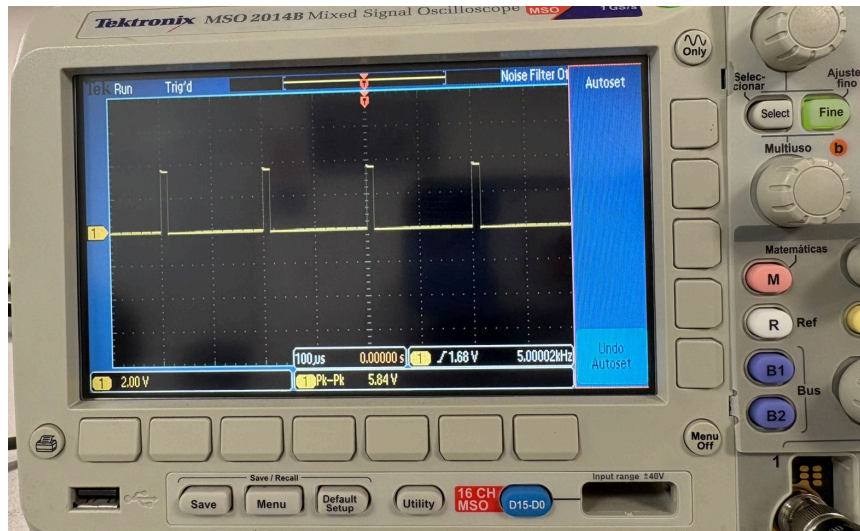


Imagen 3. Visualización del segundo caso de la señal de PWM.

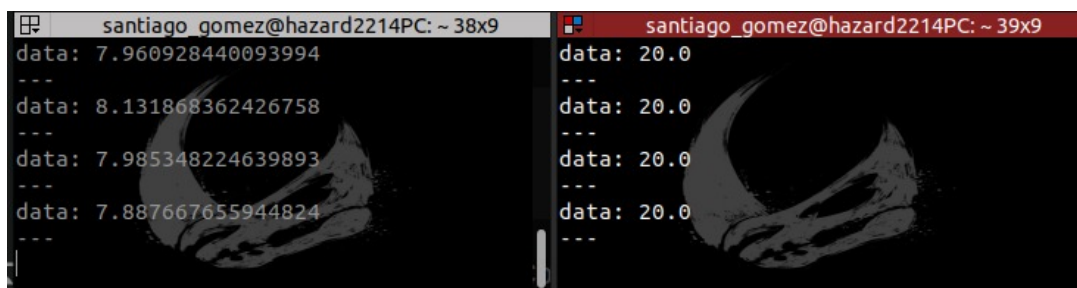
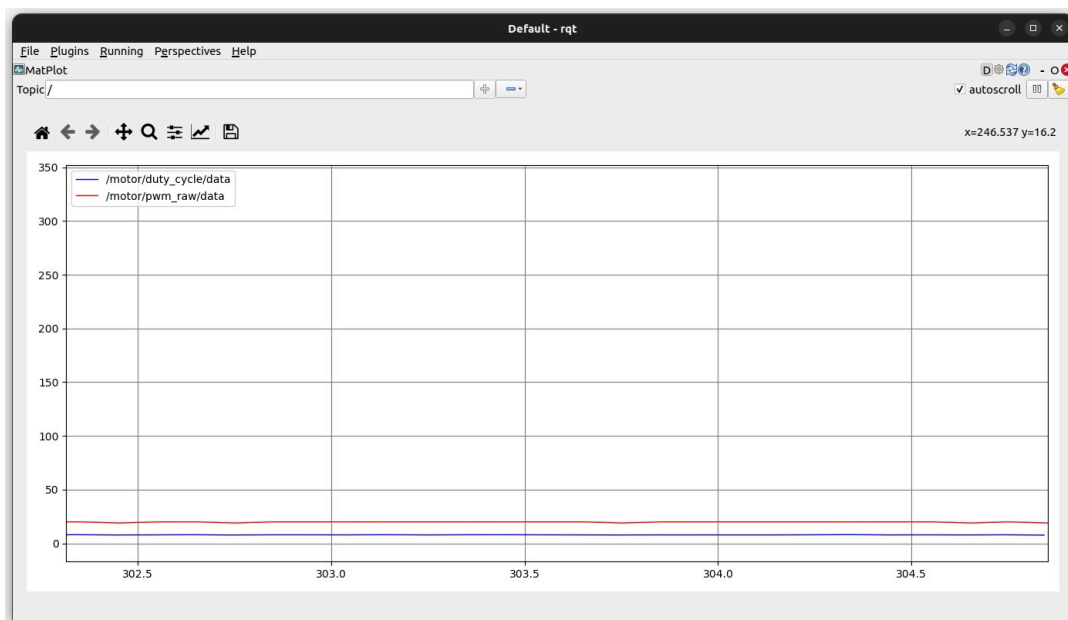


Imagen 4. Equivalencia en tópicos de ROS 2 de la señal PWM al segundo caso.

Para el segundo caso, se llevó el potenciómetro a un valor sumamente bajo para mostrar el comportamiento del motor con un duty cycle bajo. En este caso, el tiempo de la señal en

estado alto es de alrededor de $15\ \mu\text{s}$ (puede ser un valor alrededor del mismo), que corresponde a un 7.5 % de duty cycle y un valor de PWM de 19. Como se muestra en la **Imagen 4**, la gráfica muestra un duty cycle de un 8% y un PWM de 20.

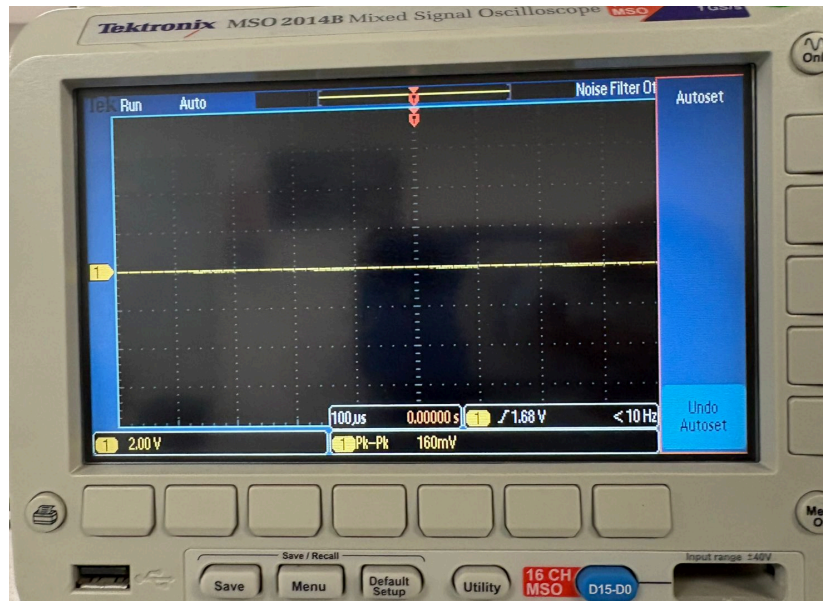


Imagen 5. Visualización del tercer caso de la señal de PWM.

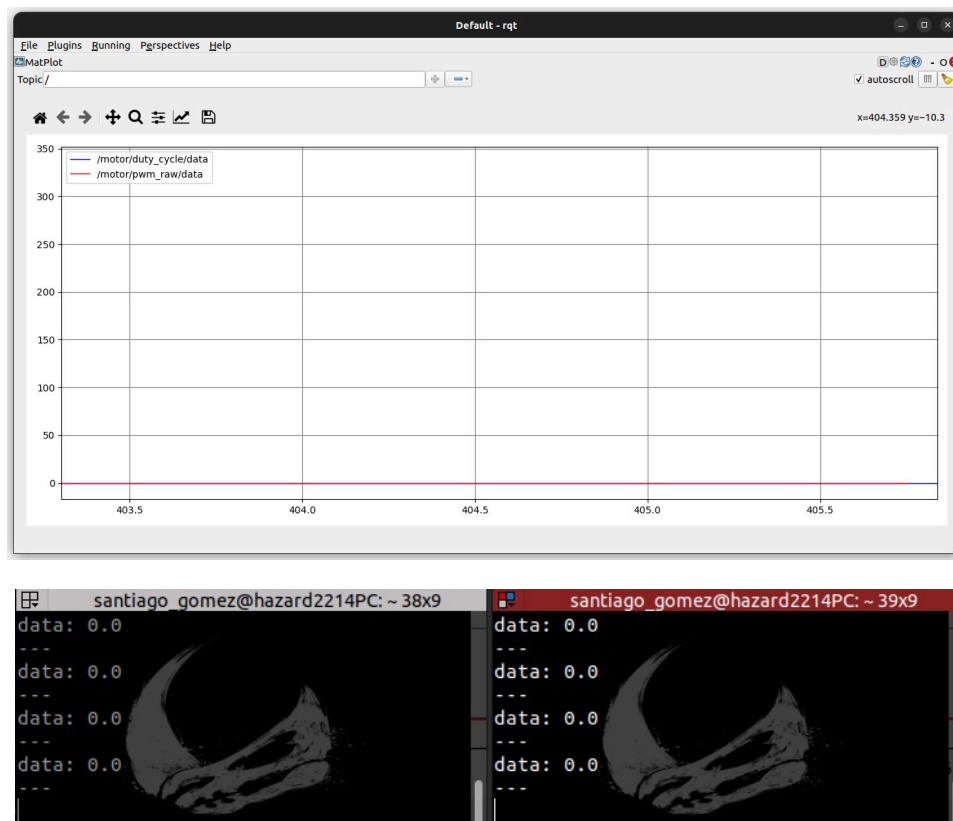


Imagen 6. Equivalencia en tópicos de ROS 2 de la señal PWM al tercer caso.

En esta ocasión, se llevó la posición del potenciómetro al mínimo para que el PWM fuera una señal que esté completamente en estado bajo. La **Imagen 6** muestra el comportamiento esperado al representar un duty cycle de 0% y un PWM con valor de 0.



Imagen 7. Visualización del tercer caso de la señal de PWM.

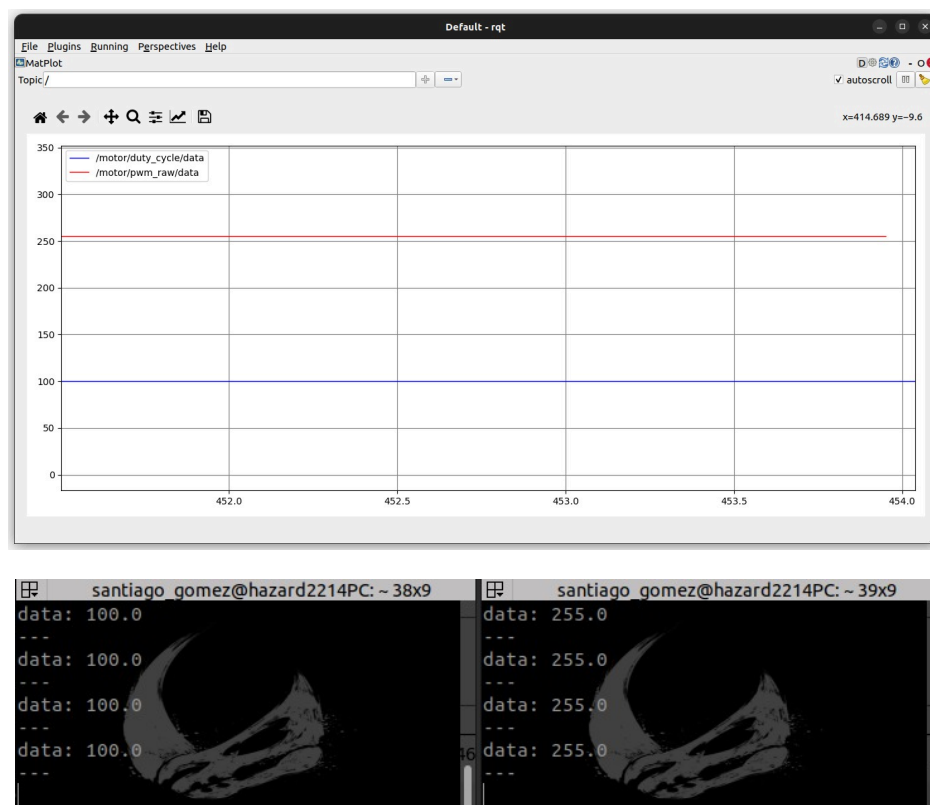


Imagen 8. Equivalencia en tópicos de ROS 2 de la señal PWM al tercer caso.

Contrariamente al tercer caso, en el cuarto caso se llevó la posición del potenciómetro a su máximo valor para hacer funcionar el motor a su máxima capacidad. Esto se puede confirmar al ver que los valores del duty cycle y del PWM están al 100% y en 255, respectivamente.

Otra manera de confirmar el funcionamiento del experimento, es al verificar la ejecución del cambio de estados mediante los tópicos `/motor/cmd` y `/motor/state`. Al publicar las letras “S” (stop), “D” (derecha) e “I” (izquierda) al primero de estos tópicos, se controla el sentido en el que gira el motor, o que se detenga por completo al publicar “S”. Dependiendo de lo que se publique, el segundo de estos tópicos mostrará una leyenda con el estado actual del motor. A continuación, se muestra el resultado:

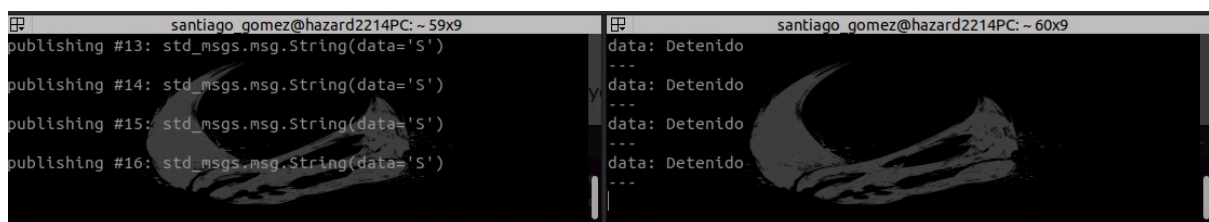


Imagen 9. Estado del motor al publicar “S”.

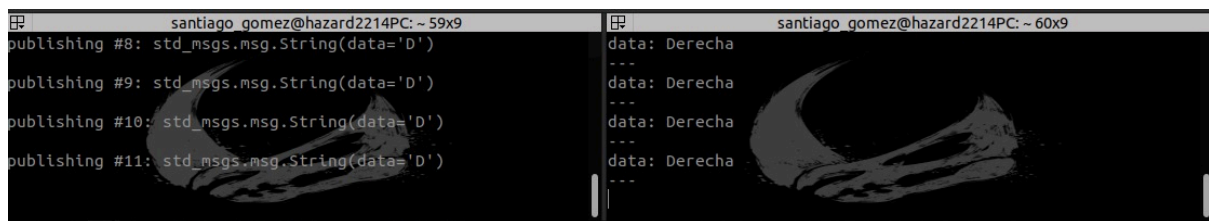


Imagen 10. Estado del motor al publicar “D”.

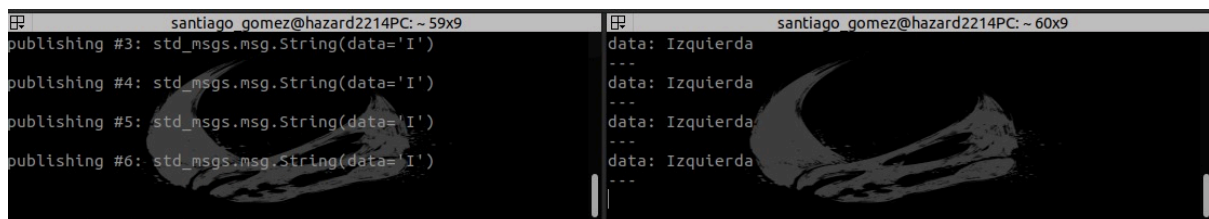


Imagen 11. Estado del motor al publicar “I”.

Como anotaciones adicionales, se determinó experimentalmente que el motor entra en su zona muerta cuando el valor de PWM es inferior a 77, lo cual corresponde aproximadamente a un duty cycle del 30 %. Por debajo de este umbral, la señal aplicada no es suficiente para vencer la fricción estática y las pérdidas internas del sistema, por lo que el motor no genera movimiento apreciable, aunque eléctricamente sí esté recibiendo una señal PWM.

Asimismo, se observó un fenómeno de saturación en valores elevados de PWM. A partir de cierto punto cercano al valor máximo, el incremento en el duty cycle ya no produce un aumento en la velocidad del motor. Esto se debe a limitaciones físicas y eléctricas del sistema, tales como el voltaje de alimentación, la resistencia interna del motor y las condiciones de carga mecánica. En esta región, el motor opera cerca de su velocidad máxima, por lo que aumentos adicionales en la señal de control no generan cambios significativos en el desempeño.

Conclusiones

Finalmente, la práctica se llevó a cabo de forma correcta y con los resultados esperados. Pues primeramente se logró una conexión entre micro-ROS y la ESP32, igualmente se implementaron los tópicos que funcionaron sin problema alguno. En este caso, lo más relevante para la práctica fue la señal PWM y su correcto funcionamiento que se pudo observar con el uso del osciloscopio del laboratorio. Y como igualmente se mencionó anteriormente, con ayuda de *rqt_plot* se puede observar el estatus en el que se encuentran los tópicos.

Finalmente, se logró que al publicar ciertos caracteres establecidos, los cuales ya se mencionaron en la sección de resultados, el estado cambiara, que físicamente podía observarse como el sentido del giro y el motor detenido. Es importante mencionar también, que los valores fueron saturados a 255, y así evitar sobrecarga del motor, tanto para el giro negativo como para el positivo, e igualmente se logró exitosamente.

Referencias

1. *Micro-ROS for ROS2 on microcontrollers*. (2022, May 4). Robotics Knowledgebase.
<https://roboticsknowledgebase.com/wiki/interfacing/microros-for-ros2-on-microcontrollers/>